

AIFP Ecosystem & Governance

Document: AIFP-DOC-14 · **Version:** 1.0.0 · **Governed by:** AIFP-1 (Doc 01)

Companion to: AIP Process (Doc 06).

AIFP aspires to be an **open standard** for autonomous-agent payments — the payment equivalent of what HTTP, TLS, and OpenAPI are for the web. This document explains why AIFP is open, how it is governed, and how implementers prove conformance.

1. Why AIFP Is Open

A payment **standard** only succeeds if no single party controls it and everyone can implement it without permission. AIFP is open because:

- **Network effects require neutrality.** Merchants and agents will only standardize on a protocol they don't have to trust a competitor to run.
- **Tokenless legitimacy.** With no native token (Whitepaper §14), there is no incentive to capture the protocol for asset value; incentives align with **usage**.
- **Permissionless implementation.** The spec (Doc 01), schemas (Doc 10), OpenAPI (Doc 08), and conformance suite are public under Apache-2.0. Anyone can build a compliant client, server, or SDK.

2. Open Standard Strategy

1. **Publish everything that defines the wire.** RFC, OpenAPI, JSON Schemas, error registry, conformance vectors — all public, all versioned.

1. **Separate protocol from product.** AiFinPay (the company) runs *a* reference implementation and commercial settlement service; the **protocol** is independent and governed in the open (AIP process).

1. **Interoperate, don't enclose.** Maintain x402 compatibility and standard stablecoins (USDC/USDT/PYUSD) rather than proprietary rails.

1. **Pursue formal standardization.** Track toward recognition by an appropriate standards

body as adoption matures (2027 roadmap).

3. Protocol Evolution

All changes flow through the **AIP process (Doc 06)**: Idea → Draft → Review → Last Call → Accepted → Final, with mandatory reference implementation + passing conformance tests before Final. SemVer governs compatibility (Doc 06 §5). Capabilities that aren't universal are gated by the **Protocol Negotiation Layer** (AIFP-1 §22.3) so old and new clients interoperate safely.

4. Governance

4.1. Bodies.

- **AIP Editors** — process gatekeepers (formatting, numbering, status).
- **Review Board** — domain maintainers (Protocol, Security, SDK, Networks) who vote on

technical merit.

- **Security Council** — must sign off on any Security-categorized AIP; can issue

emergency advisories outside the normal cadence.

- **Community** — open comment; substantive objections must be resolved.

4.2. Decision rule. Rough consensus of the Review Board, no unresolved blocking security objection; ties default to status quo. All decisions recorded publicly.

4.3. Stewardship. Over time, governance is intended to broaden beyond AiFinPay (the company) into a multi-stakeholder body (implementers, enterprises, foundations) — a deliberate move from *vendor-stewarded* to *community-stewarded* as the standard matures.

5. Community

- **Forums/Discussions** for RFCs and design debate.
 - **Public AIP repository** (Doc 15) with transparent status tracking.
 - **Contributor License:** Apache-2.0 + DCO sign-off.
 - **Code of Conduct** and clear maintainer ladders (contributor → committer → maintainer).
 - **Office hours / working groups** per domain (Protocol, Security, SDK, Networks).
-

6. Certification

An implementation may advertise "**AIFP-1 Conformant**" only if it:

1. Passes the official **conformance test suite** (§8) for its role (merchant, agent, wallet, or full).

1. Verifies receipts per AIFP-1 §7.4 (stateless, all 10 checks).
2. Honors the canonical pricing tiers, error registry, and idempotency rules.
3. Re-certifies on each protocol MINOR/MAJOR it claims to support.

A public **conformance badge + registry** lists certified implementations and the version(s) they pass.

7. Reference Implementations

- AiFinPay maintains reference **server**, **merchant SDK**, and **agent SDK**

implementations (Doc 15) under Apache-2.0.

- Reference implementations are **normative-tracking**: every Final AIP updates them.
- They are intended for learning and conformance comparison, not as the only allowed

implementation — alternative independent implementations are explicitly encouraged.

8. Compliance / Conformance Tests

The `aifinpay/conformance` repo (Doc 15) provides:

- **Protocol vectors**: canonical Payment Challenges, quotes, signed receipts (valid + intentionally invalid), nonce-replay cases, expiry/skew cases, amount-mismatch cases.

- **Role suites**: **Merchant** (must correctly 402, verify, and reject tampered receipts), **Agent** (must run the loop, respect budgets, back off correctly), **Wallet** (funding, multi-chain, budgets).

- **Negative tests**: every `AIFP-*` error must be produced under the right condition.
- **CI integration**: runnable as a GitHub Action; green required for "Conformant" status.

```
npx @aifinpay/conformance run --role merchant --base https://my-impl.example.com
-> 142 passed, 0 failed (AIFP-1 v1.0.0)
```

9. Compatibility Policy

- **MINOR** releases are backward-compatible (additive). Clients on the same MAJOR keep working.

- **MAJOR** releases require an AIP, a documented migration, and **≥12 months** parallel support of the previous MAJOR.

- **Receipts** remain verifiable within their TTL across MINOR upgrades; **kid** rotation never breaks in-flight receipts.

- **x402 compatibility** is maintained as a first-class interop surface (1,000 free migration requests).

10. Long-Term Vision

AIFP's endgame is to be **assumed infrastructure**: the default payment layer every agent framework and monetizable API speaks, governed by an open multi-stakeholder community, neutral and tokenless, settling across many chains and hybrid fiat. The two-sided flywheel — more merchants → more value for agents → more agents → more value for merchants — plus open governance and conformance certification is how AIFP compounds from a product into a **standard**.

See the Whitepaper (Doc 05 §16–18) for the strategic arc and the AIP process (Doc 06) for how the community drives it.